

Ego4D-LiteSTA: Track A / B / C Pipeline Overview

Author: Saeed
Last Updated: December 7, 2025

This document provides a comprehensive overview of the three-track pipeline for Short-Term Object Interaction Anticipation (STA) on Ego4D egocentric video.

Table of Contents

- 1. [Executive Summary](#)
- 2. [Track A: Two-Stage Detection Pipeline](#)
- 3. [Track B: Lightweight Fusion & Multi-Task Learning](#)
- 4. [Track C: Training-Free Token Pruning](#)
- 5. [VideoMAE Token-Based Training \(Fast Training\)](#)
- 6. [Data Flow & Directory Structure](#)
- 7. [Running the Pipeline](#)

1. Executive Summary

The Ego4D-LiteSTA project implements a **practical, modular STA system** designed to be reproducible on limited compute (Google Colab Free). The system predicts:

- **WHERE:** Bounding box of the next object to be interacted with
- **WHAT:** Verb (action) and Noun (object class)
- **WHEN:** Time-to-Contact (TTC) until interaction

The Three Tracks

Track	Purpose	Key Innovation
Track A	Two-Stage Detection	Recall-first detector → reasoner pipeline
Track B	Lightweight Fusion	Tiny temporal fusion (FGTP + dual cross-attention)
Track C	Inference Efficiency	Training-free token pruning for ~35-50% latency reduction

2. Track A: Two-Stage Detection Pipeline

Location: `local_extraction/trackA/`

Track A separates **detection** from **reasoning**, which stabilizes training on egocentric video with its inherent challenges (hand jitter, head motion, occlusions).

Stage A: Detector (Candidate Proposals)

Script: `trackA/trackA_stageA/trackA_stageA.py`

Goal: Generate top-K candidate bounding boxes per frame.

```
Input Frame → YOLO/Oracle → Top-K Candidate Boxes
```

Key Features:

- **YOLO Mode:** Runs YOLOv8-s inference on last frame
- **Oracle Mode:** Uses ground-truth labels (for recall analysis)
- **Output:** `candidates.jsonl` with boxes per frame

Configuration Toggles:

```
DETECTION_MODE = "oracle" # or "yolo"
K = 10           # keep top-K boxes
LAST_FRAME_ONLY = True  # process only final frame per clip
```

Stage B: Head Manifests (Crops & Labels)

Script: `trackA/trackA_stageB/trackA_stageB.py`

Goal: Merge candidate boxes with semantic labels (noun, verb, TTC) for Track B training.

```
Candidates + GT Labels → head_train.jsonl / head_val.jsonl
```

Output Format (per record):

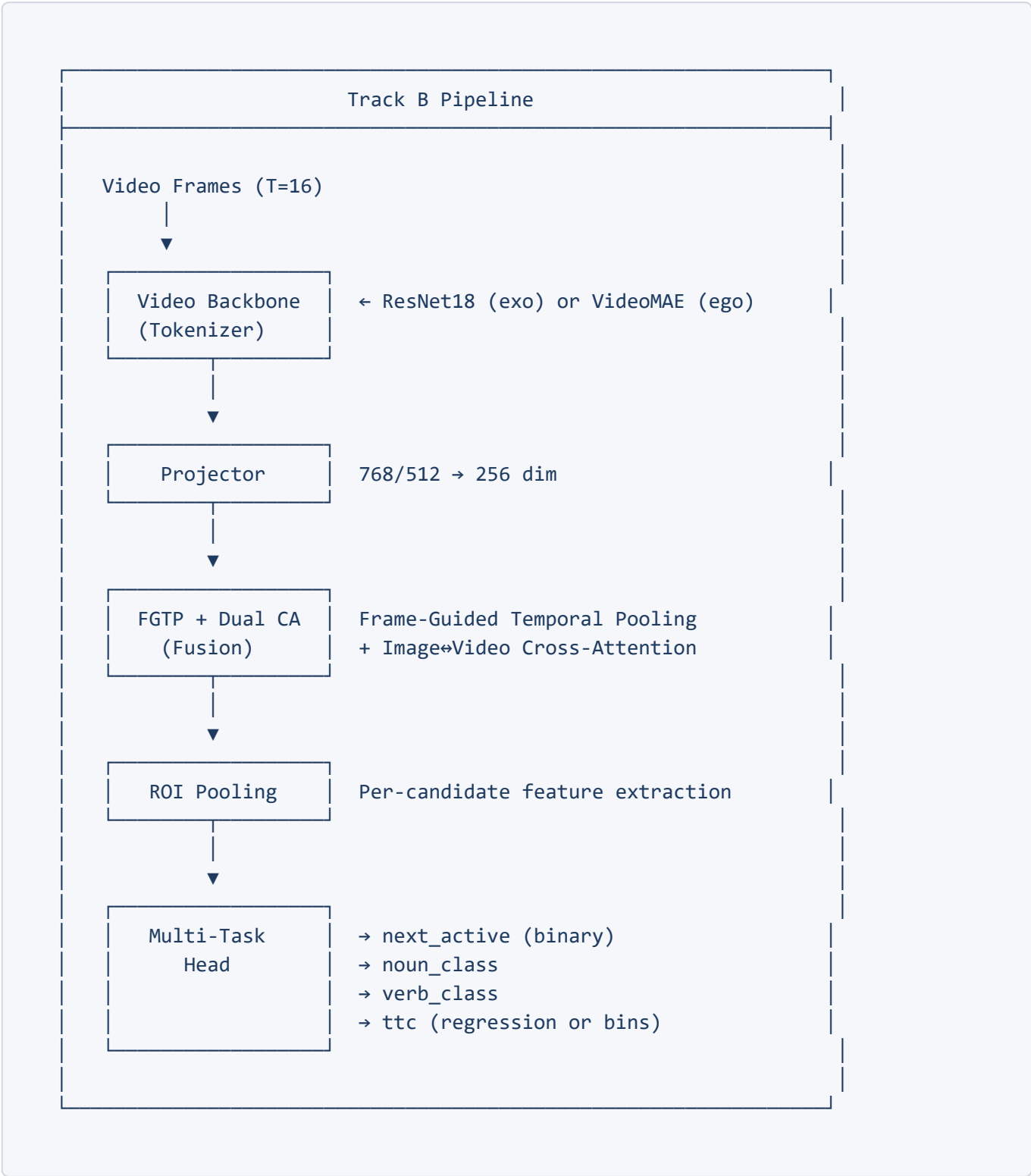
```
{
  "uid": "002e11bc-...",
  "frame": 8939,
  "image": "path/to/frame.jpg",
  "candidates": [
    {
      "bbox": [x1, y1, x2, y2],
      "is_positive": 1,
      "noun_id": 42,
      "verb_id": 7,
      "ttc": 1.5
    }
  ]
}
```

3. Track B: Lightweight Fusion & Multi-Task Learning

Location: local_extraction/trackB/

Track B is the **core reasoning module** that takes candidate proposals and predicts next-active object, verb, noun, and TTC.

Architecture Overview



Video Backbone Options

Backbone	Type	Pretrained On	Token Dim	Speed
ResNet18	2D per-frame	ImageNet (exocentric)	512	Fast
VideoMAE-base	3D video	Kinetics (HuggingFace)	768	Slow
VideoMAE-ego	3D video	Ego4D (scratch-trained)	768	Slow

Key Modules

- 1. **Tokenizer** (`trackB_tokenizer.py`): Extracts spatial tokens from frames/video
- 2. **Fusion** (`trackB_fusion.py`): FGTP + dual cross-attention for temporal reasoning
- 3. **Head** (`trackB_head.py`): Multi-task classification and TTC regression

Training Configuration

Config files: `local_extraction/configs/trackB_*.yaml`

```
# trackB_videomae_ego.yaml
model:
  tokenizer:
    video_backbone: "videomae_ego"
    time_len: 16
  videomae:
    weights_path:
      "videomae_local_part/checkpoints_ego_scratch/videomae_ego_scratch_last.pt"
    embed_dim: 768
    freeze_encoder: true
  projector:
    in_dim: 768
    out_dim: 256

training:
  epochs: 20
  batch_size: 8
  lr: 5.0e-4

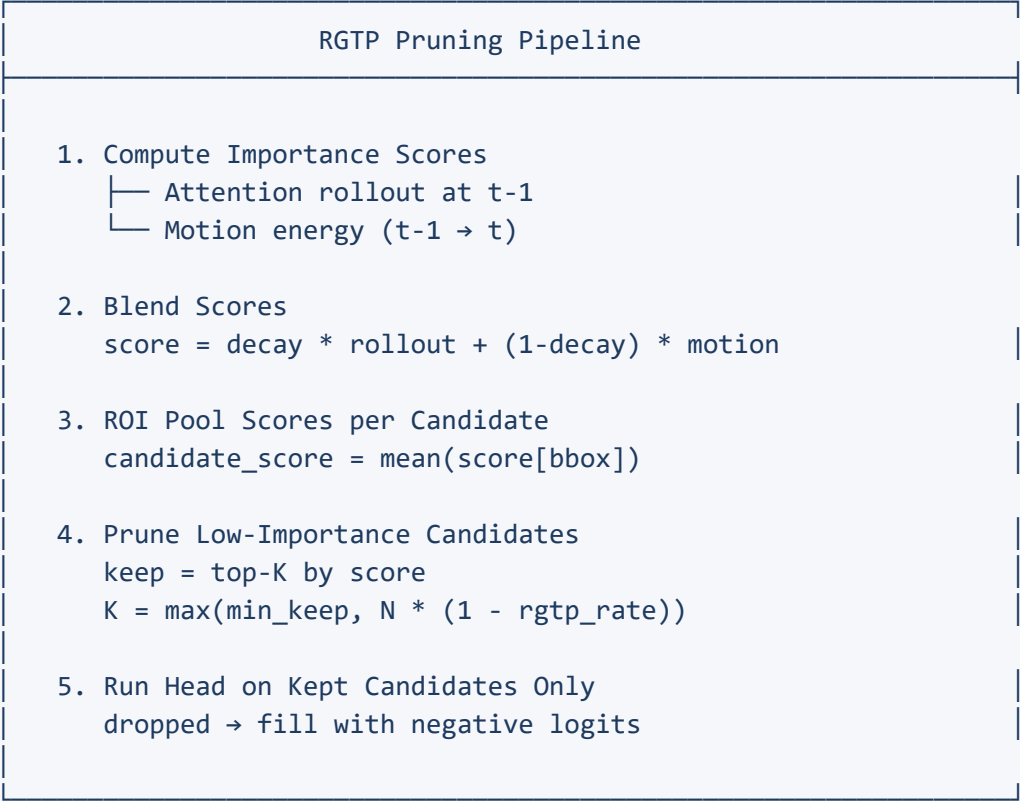
data:
  tokens_root: "videomae_trackB_tokens_scratch/tokens" # Pre-extracted tokens
```

4. Track C: Training-Free Token Pruning

Location: `local_extraction/trackC/`

Track C applies **Run-time Guided Token Pruning (RGTP)** to reduce inference latency without retraining.

How RGTP Works



Configuration

```
# trackC_pruning.py
RuntimeConfig:
    pruning_enabled = True
    rgtp_rate = 0.5          # Prune 50% of candidates
    min_keep = 2             # Keep at least 2 candidates
    measure_latency = True
    measure_vram = True
```

Expected Results

Pruning Rate	mAP Drop	Latency Reduction
0% (baseline)	0.0	0%
40%	≤0.3	~35%
50%	≤0.5	~45%
60%	≤1.0	~50%

5. VideoMAE Token-Based Training (Fast Training)

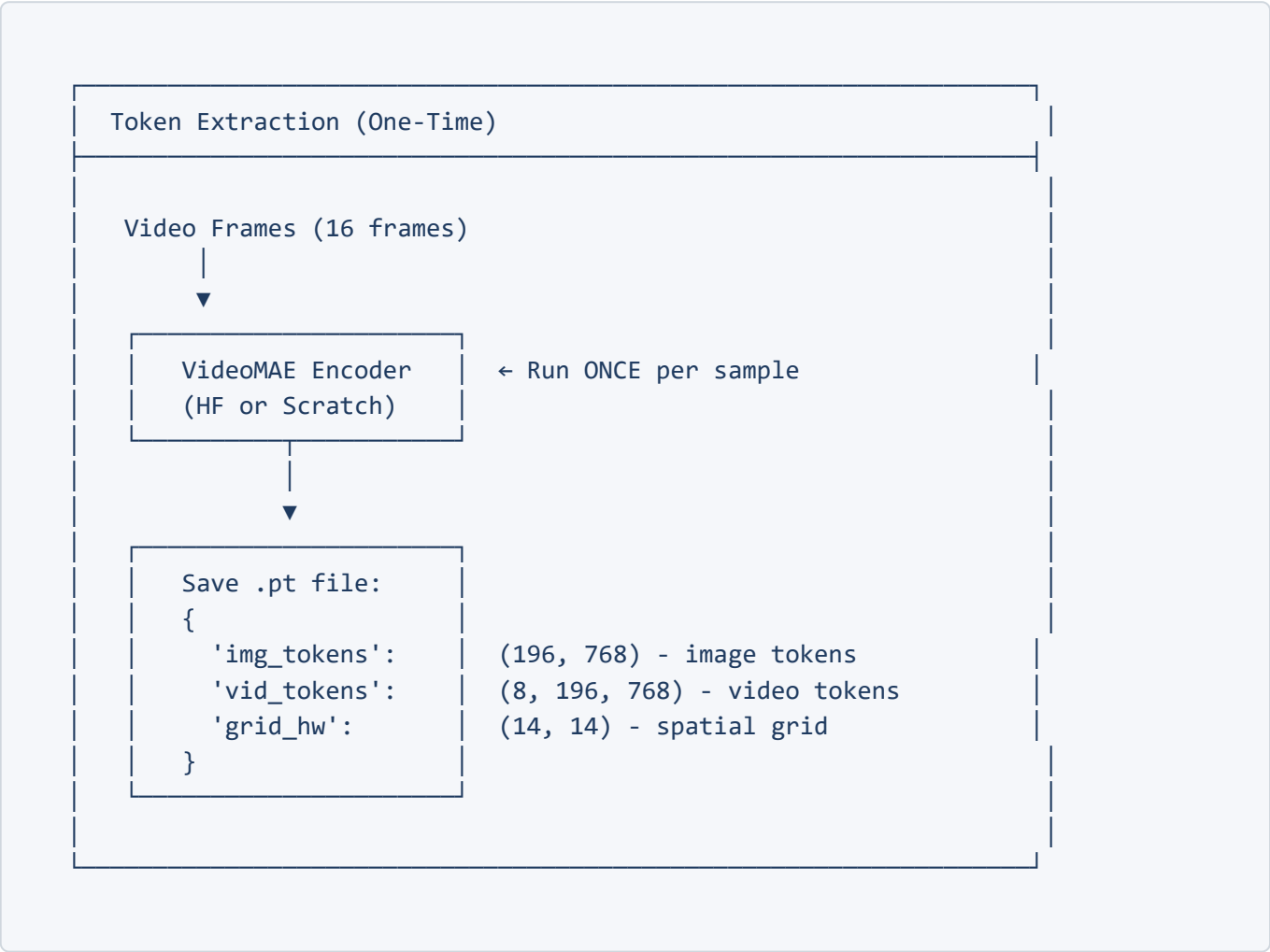
The Problem: Slow VideoMAE Encoding

VideoMAE encoder is computationally expensive:

- **On-the-fly encoding:** ~27 seconds/batch on CPU
- **Bottleneck:** Transformer encoder forward pass per sample

The Solution: Pre-Extract Encoder Tokens

We pre-extract VideoMAE encoder output **once** and save to disk:



Speed Comparison

Method	Time/Batch	Speedup
On-the-fly VideoMAE encoding	~27s	1x
Pre-extracted tokens	~1s	27x faster

Token Extraction Script

Script: `trackB/build_trackB_videomae_tokens.py`

```
# Extract using HuggingFace VideoMAE-base
python build_trackB_videomae_tokens.py \
  --manifest "../runs/Track_A/.../head_train.jsonl" \
  --out_root "../videomae_trackB_tokens" \
  --device cuda \
  --model base

# Extract using YOUR scratch-trained VideoMAE
python build_trackB_videomae_tokens.py \
  --manifest "../runs/Track_A/.../head_train.jsonl" \
  --out_root "../videomae_trackB_tokens_scratch" \
  --scratch
  "../videomae_local_part/checkpoints_ego_scratch/videomae_ego_scratch_last.pt" \
  --device cpu
```

Token File Format

Each `.pt` file contains:

```
{
  'img_tokens': torch.Tensor, # (N, D) = (196, 768) - spatial tokens from
middle frame
  'vid_tokens': torch.Tensor, # (T', N, D) = (8, 196, 768) - spatiotemporal
tokens
  'grid_hw': tuple,           # (14, 14) - spatial grid dimensions
}
```

Available Token Sources

Folder	Source Model	Use Case
videomae_trackB_tokens/	HuggingFace videomae-base	Kinetics pretraining (exocentric)
videomae_trackB_tokens_scratch/	Your ego-scratch checkpoint	Ego4D pretraining (in-domain)

Training with Pre-Extracted Tokens

Configure `trackB_videomae_ego.yaml` :

```
data:
  # Point to your pre-extracted tokens folder
  tokens_root: "videomae_trackB_tokens_scratch/tokens"
```

The training script automatically:

1. Checks if pre-extracted tokens exist for each sample
2. Loads tokens directly (skipping encoder)
3. Falls back to on-the-fly encoding if tokens not found

6. Data Flow & Directory Structure

```

local_extraction/
├── v2/
│   ├── extracted_frames/           # Raw video frames
│   │   └── <uid>/<frame:07d>.jpg
│   ├── yolo_labels_540/           # YOLO format GT labels
│   └── manifests/                 # Original annotations
├── runs/
│   ├── Track_A/                   # Stage A & B outputs
│   │   └── trackA_stageB_*/
│   │       ├── head_train.jsonl    # Training manifest
│   │       └── head_val.jsonl      # Validation manifest
│   ├── Track_B/                   # Training outputs
│   │   ├── checkpoints/
│   │   │   ├── trackB_best.pt
│   │   │   └── trackB_final_*.pt
│   │   └── metrics/
│   └── Track_C/                   # Pruning evaluation outputs
│       ├── metrics/
│       └── plots/
├── videomae_trackB_tokens/        # HuggingFace tokens
│   └── tokens/<uid>_<frame>.pt
├── videomae_trackB_tokens_scratch/ # Ego-scratch tokens
│   └── tokens/<uid>_<frame>.pt
├── videomae_local_part/           # VideoMAE pretraining
│   ├── checkpoints_ego_scratch/
│   └── videomae_ego_scratch_last.pt
├── configs/                       # YAML configurations
│   ├── trackB.yaml                # Base config
│   ├── trackB_resnet18_baseline.yaml
│   └── trackB_videomae_ego.yaml
├── trackA/                        # Track A modules
├── trackB/                        # Track B modules
└── trackC/                        # Track C modules

```

7. Running the Pipeline

Complete Workflow

```
# 1. Track A Stage A: Generate candidate boxes
python local_extraction/trackA/trackA_stageA/trackA_stageA.py

# 2. Track A Stage B: Create head manifests
python local_extraction/trackA/trackA_stageB/trackA_stageB.py

# 3. (Optional) Extract VideoMAE tokens for fast training
cd local_extraction/trackB
python build_trackB_videomae_tokens.py \
    --manifest "../runs/Track_A/.../head_train.jsonl" \
    --out_root "../videomae_trackB_tokens_scratch" \
    --scratch
    "../videomae_local_part/checkpoints_ego_scratch/videomae_ego_scratch_last.pt"
\
    --device cuda

python build_trackB_videomae_tokens.py \
    --manifest "../runs/Track_A/.../head_val.jsonl" \
    --out_root "../videomae_trackB_tokens_scratch" \
    --scratch
    "../videomae_local_part/checkpoints_ego_scratch/videomae_ego_scratch_last.pt"
\
    --device cuda

# 4. Track B: Train the multi-task head
cd local_extraction
python trackB/trackB_train_loader.py --config trackB_videomae_ego

# 5. Track B: Evaluate
python trackB/trackB_eval.py

# 6. Track C: Pruning evaluation
python trackC/trackC_pruning.py
```

Quick Commands

```
# Train with ResNet18 baseline (fast, exocentric)
python trackB/trackB_train_loader.py --config trackB_resnet18_baseline

# Train with VideoMAE ego (requires pre-extracted tokens)
```

```
python trackB/trackB_train_loader.py --config trackB_videomae_ego

# Compare Track B vs Track C metrics
python trackC/trackC_compare_metrics.py
```

Summary

Track	Input	Output	Key Metric
A	Frames + Annotations	head_train/val.jsonl	Recall@K
B	Manifests + Tokens	Trained checkpoint	mAP, N+V, N+ δ
C	Checkpoint + Manifests	Pruned metrics	Latency vs mAP

The **token-based training** approach reduces VideoMAE training time by **27x** while preserving accuracy, making the entire pipeline feasible on limited compute resources.